CMCMIS_SIMPLIFIED

PHASE 9 HOTFIX — FULLY SEALED TECHNICAL DOCUMENTATION

Maintenance + Spares CRUD · Date Input Persistence · Mark-Complete Gate Refresh · Closure Button Enablement

SEALED / LOCKED / DS DOCUMENTATION STYLE

LOCKED DOCUMENT PURPOSE

This document explains the Phase 9 hotfix work as a senior technical documentation package. It focuses on why the bugs appeared, how the logic should be understood, how database/backend/frontend layers were corrected, and how the final behavior was smoke-verified. It intentionally avoids dumping full source code; only small logic snippets are included where useful.

Field	Value
Project	CMCMIS_SIMPLIFIED — Computerized Maintenance & Calibration Management Information System
Phase covered	Phase 9 Hotfix after Job Card Detail + Observations + Engineer completion
Source input	Uploaded pasted implementation transcript
Primary fixed areas	Maintenance Details CRUD, Spares Used CRUD, date-field persistence, Mark Complete stale-data gate, Closure tab disabled button
Documentation mode	Technical documentation engineer style; senior developer reasoning; step-by-step; DS-style locked report
Generated on	2026-05-19 13:30 UTC

Static Table of Contents

DOCUMENT NOTE

This is a static TOC for a sealed engineering record. Word automatic TOC fields are intentionally avoided to keep the document deterministic across environments.

1. Executive Lock Summary

ONE-BLOCK FINAL SUMMARY

Phase 9 was already functional at the Job Card lifecycle level, but post-implementation testing exposed four practical UI/data-flow bugs: two sub-tabs were migrated but not fully exposed as CRUD modules, datetime fields were returned in ISO format that date inputs could not render, the Mark Complete gate could read stale observations immediately after save, and the Closure button used the wrong write-gate for COMPLETED state. The hotfix sealed these gaps by shipping Maintenance and Spares CRUD end-to-end, normalizing datetime values for date inputs, forcing immediate detail refetch after tab saves, and overriding per-tab write access so Closure uses canVerifyClose instead of the generic IN_PROGRESS-only canWrite flag.

Executive Bug/Fix/Acceptance Matrix

Bug ID	Surface	Root Cause	Fix	Acceptance
P9-HF-01	Maintenance Details tab	DB table existed but no real CRUD endpoint + FE row UI	New BE sub-module, routes, FE hooks/API, real multi-row UI	Add/list/patch/delete maintenance row passed
P9-HF-02	Spares Used tab	DB table existed but no real CRUD endpoint + FE row UI	New BE sub-module, routes, FE hooks/API, real multi-row UI	Add/list/patch/delete spare row passed
P9-HF-03	Date inputs	BE returned ISO datetime into HTML date input	coerceFieldForInput truncates ISO to YYYY-MM-DD	Date round-trip and empty-to-null passed
P9-HF-04	Mark Complete	Detail payload was stale after fast tab switch	invalidateAll + explicit refetch after save	Gate sees fresh observations immediately
P9-HF-05	Closure	Closure tab used generic canWrite requiring IN_PROGRESS	Per-tab write-gate: closure → canVerifyClose	Button enables for LIC/SA when status=COMPLETED

2. Original Phase 9 Context

Phase 9 baseline

Phase 9 created the Job Card Detail + Observations + Engineer workflow. It introduced the 13-tab Job Card page, lifecycle transitions, task checklist, documents, mark-complete gates, closure/reopen logic, and job-card state history. The hotfix does not redesign Phase 9; it completes and hardens practical gaps discovered during

browser testing.

Why hotfix was necessary

In production-grade software, a phase can pass architectural smoke tests but still reveal UI-state bugs when a real engineer flows quickly through tabs. These bugs are valuable because they expose the difference between server capability and usable workflow continuity.

Locked doctrine preserved

No schema-damaging redesign was introduced. The hotfix follows the same doctrine: modular routes, repo/service/controller separation, row-level ownership gates, frontend hooks wrapping API calls, and deterministic smoke checks before declaring sealed completion.

Layer	Phase 9 baseline	Hotfix contribution
Database	jc_maintenance_details and jc_spares_used tables already existed	Hotfix uses those tables instead of creating new duplicate tables
Backend	Job Card routes existed but child tabs were incomplete	Hotfix adds maintenance and spares submodules with full CRUD
Frontend	13-tab UI existed	Hotfix replaces stub tab messages with row-based editing tables
State logic	Mark Complete and Closure transitions existed	Hotfix corrects stale gate data and wrong tab write-access
QA	Phase 9 smoke existed	Hotfix adds targeted H1-H9 smoke matrix

3. Hotfix Problem Register

Category	Observation
Problem	User could open Job Card detail but Maintenance and Spares tabs were not real data-entry modules.
Problem	Date fields appeared blank after save, creating the impression that the backend had not persisted data.
Problem	Mark as Complete could remain disabled even when the user had just entered observations.
Problem	Closure tab could appear but Close Job Card stayed disabled for the LIC.

Category	Observation
Impact	These issues blocked real end-to-end Job Card completion from engineer to LIC closure.

4. Bug 1 — Maintenance Details + Spares Used Were Stubs

ROOT CAUSE

The migration created jc_maintenance_details and jc_spares_used, but the application had not yet exposed these tables through production routes and frontend CRUD components. This created a classic mismatch: persistence existed, but the product workflow could not use it.

LOCKED FIX

The hotfix created two parallel backend submodules, mounted them under /job-cards/:id/maintenance-rows and /job-cards/:id/spares-rows, added frontend API wrappers and hooks, then replaced the tabs with editable row grids.

ACCEPTANCE SIGNAL

H1-H6 smoke checks added, listed, patched, and deleted both maintenance and spare rows successfully.

Flow diagram

```
Engineer opens tab → Add Row → POST child row → Row appears → Edit cell → PATCH row field → Refetch → Trash → DELETE row
```

Before	After
Functional surface existed partially; user flow got blocked.	Functional surface is complete and aligned with the actual Job Card lifecycle.
User had to guess what was missing or wait for polling.	UI now either persists immediately or shows clear current-vs-required gate hints.
Bug was discovered only by realistic user navigation.	Fix is encoded into shared hooks and per-tab gates to prevent recurrence.

7. Bug 2 — Date Input Could Not Show Saved Values

ROOT CAUSE

Backend correctly returned datetime columns as ISO strings, for example 2026-04-10T00:00:00.000Z. The HTML input type=date only accepts YYYY-MM-DD. Browser silently dropped the value, showing an empty input.

LOCKED FIX

The shared tab form hook now converts known datetime-as-date fields by slicing the first 10 characters

before binding them to date inputs.

ACCEPTANCE SIGNAL

H7–H9 confirmed YMD date round-trip, datetime wire format starts with expected date, and empty date clears to NULL.

Flow diagram

```
BE ISO datetime → FE coerceFieldForInput → YYYY-MM-DD → input[type=date] renders correctly
```

Before	After
Functional surface existed partially; user flow got blocked.	Functional surface is complete and aligned with the actual Job Card lifecycle.
User had to guess what was missing or wait for polling.	UI now either persists immediately or shows clear current-vs-required gate hints.
Bug was discovered only by realistic user navigation.	Fix is encoded into shared hooks and per-tab gates to prevent recurrence.

8. Bug 3 — Mark Complete Button Disabled Due to Stale Detail Payload

ROOT CAUSE

The page invalidated cache after save but did not force immediate refetch. If a user saved observations and switched to Mark Complete quickly, the gate read the previous detail payload.

LOCKED FIX

The tab save hook now performs invalidateAll() plus explicit refetch(), so the parent Job Card detail payload refreshes immediately after manual or auto-save.

ACCEPTANCE SIGNAL

FE build passed; gate panel now shows current/required hints and a direct Open Observations link.

Flow diagram

```
Save observations → PATCH 200 → invalidateAll + refetch → fresh jc.observations_text → gate green
```

Before	After
Functional surface existed partially; user flow got blocked.	Functional surface is complete and aligned with the actual Job Card lifecycle.
User had to guess what was missing or wait for polling.	UI now either persists immediately or shows clear current-vs-required gate hints.

Before	After
Bug was discovered only by realistic user navigation.	Fix is encoded into shared hooks and per-tab gates to prevent recurrence.

9. Bug 4 — Closure Button Disabled Due to Wrong Write Gate

ROOT CAUSE

The generic canWrite flag was correct for IN_PROGRESS data tabs, but Closure is used in COMPLETED status. Passing canWrite into Closure made the button disabled exactly when it should be usable.

LOCKED FIX

The orchestrator now resolves activeTabCanWrite per tab: data tabs use canWrite, Mark Complete uses canMarkComplete, Closure uses canVerifyClose.

ACCEPTANCE SIGNAL

FE build passed; Closure button now enables for LIC/SA when status=COMPLETED and form is valid.

Flow diagram

```
COMPLETED status → Closure visible → activeTabCanWrite = canVerifyClose → form valid → Close Job Card enabled
```

Before	After
Functional surface existed partially; user flow got blocked.	Functional surface is complete and aligned with the actual Job Card lifecycle.
User had to guess what was missing or wait for polling.	UI now either persists immediately or shows clear current-vs-required gate hints.
Bug was discovered only by realistic user navigation.	Fix is encoded into shared hooks and per-tab gates to prevent recurrence.

5. Backend Fix Architecture

BACKEND THINKING MODEL

Do not patch UI first when the database has missing write surfaces. First expose a clean resource model, then let frontend tabs become simple consumers. Maintenance and Spares are child resources of a Job Card, so their routes must be nested below the Job Card identity.

Backend file family	Responsibility	Reasoning
maintenance.repo.js	SQL-only DAL for jc_maintenance_details	Keeps table knowledge isolated and prevents SQL from leaking

Backend file family	Responsibility	Reasoning
		into service/controller layers.
maintenance.service.js	Authorization + business logic	Checks Job Card existence, legacy read-only status, and engineer/LIC ownership before mutation.
maintenance.controller.js	HTTP request/response shaping	Converts params/body/user into service input and returns stable JSON envelopes.
maintenance.routes.js	Route definitions + middleware	Applies authenticate, authorize, validate before controller execution.
spares.repo.js	SQL-only DAL for jc_spares_used	Handles decimal normalization and row ordering by sr_no.
spares.service.js	Authorization + business logic	Mirrors maintenance service for predictable behavior.
spares.controller.js	HTTP shaping	Same shape as maintenance controller to reduce cognitive load.
spares.routes.js	Nested routes	Mounted under Job Card to preserve parent-child semantics.

5.1 Backend Route Contract

Endpoint	Method	Purpose	Write gate
/job-cards/:id/maintenance-rows	GET	List maintenance details rows	job_card:read-detail
/job-cards/:id/maintenance-rows	POST	Add maintenance detail row	job_card:update-tasks + own engineer/LIC/SA
/job-cards/:id/maintenance-rows/:rowId	PATCH	Edit one maintenance row	job_card:update-tasks + own engineer/LIC/SA
/job-cards/:id/maintenance-	DELETE	Hard-delete one maintenance row	job_card:update-tasks + own engineer/LIC/SA

Endpoint	Method	Purpose	Write gate
rows/:rowId			
/job-cards/:id/spares-rows	GET	List spare rows	job_card:read-detail
/job-cards/:id/spares-rows	POST	Add spare row	job_card:update-tasks + own engineer/LIC/SA
/job-cards/:id/spares-rows/:rowId	PATCH	Edit one spare row	job_card:update-tasks + own engineer/LIC/SA
/job-cards/:id/spares-rows/:rowId	DELETE	Hard-delete one spare row	job_card:update-tasks + own engineer/LIC/SA

Light endpoint map

```
GET /api/v1/job-cards/:id/maintenance-rows
POST /api/v1/job-cards/:id/maintenance-rows
PATCH /api/v1/job-cards/:id/maintenance-rows/:rowId
DELETE /api/v1/job-cards/:id/maintenance-rows/:rowId


GET /api/v1/job-cards/:id/spares-rows
POST /api/v1/job-cards/:id/spares-rows
PATCH /api/v1/job-cards/:id/spares-rows/:rowId
DELETE /api/v1/job-cards/:id/spares-rows/:rowId
```

5.2 Backend Authorization Doctrine

Condition	Outcome	Why
Job Card not found	404	A child row cannot exist logically without a valid parent card.
Legacy Job Card	409 conflict	Legacy imported rows are read-only to prevent corrupting historic data.
Assigned engineer	Allowed to mutate own Job Card rows	Engineer owns operational data entry.
Lab In-Charge / Super Admin	Allowed to mutate	Supervisory override for operational correction.
Other engineer	403	Prevents cross-assignment edits.

6. Frontend Fix Architecture

FRONTEND THINKING MODEL

A production tab should not be a static form island. It should have a clear data hook, API wrapper, row-local

draft state, refetch/invalidation discipline, disabled state, and user-visible error surface. The hotfix introduced that pattern for Maintenance and Spares.

Frontend artifact	Responsibility	Why it matters
lib/api/jobCards.js	Adds wrappers for maintenance and spares endpoints	Centralizes HTTP paths and envelope unwrapping.
useMaintenanceRows.js	Fetch/cache/refetch maintenance child rows	Prevents each component from reinventing fetch state.
useSparesRows.js	Fetch/cache/refetch spare child rows	Separates spares cache from maintenance cache.
MaintenanceDetailsTab.jsx	Editable multi-row table	Real engineer-facing CRUD UI.
SparesUsedTab.jsx	Editable multi-row table with source enum and decimal fields	Real spare-entry workflow with cost/quantity behavior.
SimpleDataTabs.jsx	Shared tab form save/refetch behavior	Fixes stale detail payload across all simple tabs.
MarkCompleteTab.jsx	Gate panel and completion submit	Improves observability of why button is disabled.
JobCardDetail.jsx	Per-tab canWrite resolution	Prevents status-gate bugs across tabs.

6.1 Row Editing Pattern

Frontend row CRUD mental model

```
User clicks Add Row
  → POST child row
  → invalidate child-row cache
  → refetch rows
  → row appears


User edits one cell
  → local draft updates immediately
  → onBlur triggers PATCH for one field only
  → refetch rows
  → table shows saved value


User deletes row
  → confirm dialog
  → DELETE child row
  → refetch rows
```

UX event	Network call	Why this pattern is safe
Add Row	POST	Creates a database row before inline editing begins.

UX event	Network call	Why this pattern is safe
Edit-on-blur	PATCH one field	Avoids noisy requests on every keystroke while preserving quick save behavior.
Select source change	PATCH immediately	Dropdown selection is atomic; no need to wait for blur.
Delete	DELETE after confirm	Hard-delete is explicit and cannot happen accidentally.

7. Date Input Persistence Logic

WHY THIS BUG FELT LIKE DATABASE FAILURE

The database was not the failure. The value existed on the wire, but the browser would not render it into input[type=date]. This is dangerous because the user sees an empty field and assumes persistence failed. The correct mental model is: storage can be correct while UI rendering is wrong.

Field family	Wire example	HTML input expectation	Fix
DATETIME rendered as date	2026-04-10T00:00:00.000Z	YYYY-MM-DD	Slice to first 10 characters before binding
DATE-only columns	2026-04-10	YYYY-MM-DD	No conversion needed
Empty value	NULL or empty string	empty UI input	Convert empty string to NULL server-side and show blank client-side

Datetime-to-date conversion principle

```
Server returns:   2026-04-10T00:00:00.000Z
Browser date input accepts only: 2026-04-10

Fix: displayValue = raw.slice(0, 10) for DATETIME fields used as dates
```

8. Mark Complete Gate Logic

MARK COMPLETE IS A GATED TRANSITION, NOT A BUTTON STYLE ISSUE

The Mark as Complete button should only enable when the engineering record is complete enough to survive audit. The bug was not that the button had the wrong disabled formula; the bug was that the gate formula could read stale parent data. The hotfix keeps the gate strict while making the data fresh.

Gate	Condition	User guidance after hotfix

Gate	Condition	User guidance after hotfix
All tasks completed	0 tasks or completed count equals total count	Shows x/y complete and Open Task Checklist link.
Observations recorded	observations_text length >= 20 characters	Shows current/20 characters and Open Observations link.
Calibration certificate	Required for calibration workflows	Shows upload requirement and Open Documents link.
Required document	At least one required-tier document	Shows document count and Open Documents link.

Stale-data race fix

```
Before:
  PATCH observation → invalidate cache → wait up to 15 seconds for poll

After:
  PATCH observation → invalidate cache → explicit refetch now → gate reads fresh value
```

9. Closure Write-Gate Logic

CLOSURE NEEDED A DIFFERENT STATUS GATE

The data-entry tabs are editable in IN_PROGRESS. Closure is editable in COMPLETED. Reusing one canWrite flag for all tabs looked clean but was semantically wrong. The sealed fix resolves write access per active tab.

Tab class	Correct status	Correct write flag
Simple data tabs	IN_PROGRESS	canWrite
Task Checklist	IN_PROGRESS	canWrite
Documents	IN_PROGRESS	canWrite
Mark Complete	IN_PROGRESS	canMarkComplete
Closure	COMPLETED	canVerifyClose

Light code: per-tab write gate

```
let activeTabCanWrite = canWrite;
if (activeTab === 'closure')       activeTabCanWrite = canVerifyClose;
if (activeTab === 'mark-complete') activeTabCanWrite = canMarkComplete;
```

10. How to Think for Logic Development

Step 1 — Identify the real blocking surface

Do not assume the button is broken. First inspect all gates, network state, backend response, and status

transitions. In this hotfix, each bug had a different surface: missing child resource, bad UI value coercion, stale cache, and wrong status gate.

Step 2 — Classify the bug by layer

Maintenance/Spares = missing backend/frontend resource surface. Date = frontend value coercion. Mark Complete = frontend cache invalidation. Closure = orchestrator permission/status logic.

Step 3 — Fix at the lowest correct layer

Do not hack the button disabled prop if the parent gate is wrong. Do not alter database schema if the UI merely cannot render ISO timestamps. Put the fix where the broken assumption lives.

Step 4 — Encode the pattern, not only the symptom

The per-tab canWrite fix becomes a reusable pattern for future Phase 11 PDF tabs and report tabs. The date coercion fix lives in the shared form hook, so future simple tabs inherit it.

Step 5 — Verify by behavior, not confidence

A fix is not sealed because code compiles. It is sealed when smoke tests prove the exact broken workflow now passes.

11. Database Reasoning and Persistence Shape

Table	Role in hotfix	Important behavior
jc_maintenance_details	Stores multi-row maintenance/defect details	Rows ordered by sr_no; hard-delete supported.
jc_spares_used	Stores spare parts used in repair/calibration	Decimal cost/quantity normalized; source enum default.
cmms_jobcard_mst	Stores parent Job Card and simple tab fields	Datetime fields can return ISO wire values.
jc_documents	Used by Mark Complete gates	Required/cert document checks depend on doc_type.
jc_task_checklist	Used by Mark Complete gates	All completed or no tasks required to pass.

NO NEW TABLES REQUIRED

The hotfix did not create duplicate child tables. It used the Phase 9 migration tables already designed for these exact tab surfaces. This preserves database integrity and avoids schema drift.

12. Backend Layer-by-Layer Reasoning

Layer	Maintenance example	Spares example	General lesson
Route	/:id/maintenance-rows	/:id/spares-rows	Nested resource path

Layer	Maintenance example	Spares example	General lesson
			makes the parent Job Card explicit.
Validator	maintenanceRowSchema	spareRowSchema	Accept only allowed field shapes before business logic.
Controller	postAddRow / patchRow / deleteRow	postAddRow / patchRow / deleteRow	Keep HTTP envelope logic thin.
Service	loadAndAuthorize + repo call	loadAndAuthorize + repo call	Business rules and authorization live here.
Repo	list/insert/update/delete	list/insert/update/delete	SQL is isolated and testable.

Backend request pipeline

```
Route → authenticate → authorize → validate → controller → service → repo → DB
Response ← JSON envelope ← service output ← repo result
```

13. Frontend Layer-by-Layer Reasoning

Layer	Responsibility	Why it is production-grade
API wrapper	Encodes endpoint path and returns unwrapped data	Components are not polluted with Axios envelope details.
Hook	Owns loading/error/refetch/cache	Tabs remain focused on UI behavior.
Tab component	Owns row-local draft state and user interaction	Prevents incomplete network state from breaking typing.
Orchestrator	Determines active tab and write gates	Central place for status + permission semantics.
Shared form hook	Normalizes values and saves tab payloads	One fix updates all simple tabs.

IMPORTANT FRONTEND PRINCIPLE

The tab component is not the source of truth; the database is. The tab may hold local draft values while the user is typing, but after save/refetch, the server snapshot rehydrates the UI.

14. RBAC and State Integrity

Workflow action	Allowed actor	Required status	Hotfix protection

Workflow action	Allowed actor	Required status	Hotfix protection
Edit Maintenance rows	Assigned engineer or LIC/SA	IN_PROGRESS parent context	Service checks legacy + ownership.
Edit Spares rows	Assigned engineer or LIC/SA	IN_PROGRESS parent context	Service checks legacy + ownership.
Mark Complete	Assigned engineer or LIC/SA	IN_PROGRESS	Gate panel and backend pre-completion checks.
Verify Close	LIC/SA	COMPLETED	Closure tab uses canVerifyClose.
Read legacy cards	Permitted roles	Any legacy status	Read-only banner and mutation blocking.

State-write alignment

```
ASSIGNED → IN_PROGRESS → COMPLETED → VERIFIED_CLOSED
           data tabs       mark complete      closure
           canWrite        canMarkComplete    canVerifyClose
```

15. Smoke Verification and Acceptance Evidence

Smoke	Expected behavior	Result
H1	Add maintenance row returns id and sr_no	PASS
H2	List maintenance rows returns new row	PASS
H3	Patch maintenance row persists changed remarks	PASS
H4	Add empty spare row returns id with default source	PASS
H5	Patch spare row persists enum and decimal cost	PASS
H6	Delete both rows successfully	PASS
H7	DATE field round-trips YYYY-MM-DD	PASS
H8	DATETIME field returns ISO but frontend truncates to YYYY-MM-DD	PASS

Smoke	Expected behavior	Result
H9	Empty date clears to NULL	PASS
FE Build	Vite production build completes	PASS

SEALED ACCEPTANCE RESULT

Hotfix smoke produced ALL CHECKS PASSED for 10 checks, and frontend production build completed successfully. This means both server behavior and browser bundle integrity were verified.

Acceptance output summary

```
ALL 10 CHECKS PASSED
FE build clean: 1744 modules transformed
Generated bundle validated after closure and mark-complete fixes
```

16. Locked Final Behavior

User flow	Now expected behavior
Engineer opens Maintenance Details	Real grid appears with Add Row button and inline editable fields.
Engineer adds maintenance row	Row is persisted, returned with sr_no, displayed in table.
Engineer edits maintenance row	Tab-out triggers PATCH and saved value remains after refetch.
Engineer opens Spares Used	Real grid appears with source dropdown, quantity and cost fields.
Engineer enters date fields	Saved values reappear correctly in date inputs.
Engineer adds observations then switches tabs	Parent detail refetches immediately; Mark Complete sees fresh data.
Engineer reaches Mark Complete	Gate panel shows exact missing items with jump links.
LIC opens Closure on completed card	Closure form is editable; Close button enables when form is valid.

FINAL LOCK STATEMENT

The hotfix converts Phase 9 from architecturally complete to operationally complete for the tested Job Card completion path: data entry, row-based child records, observation gating, mark-complete transition, and LIC closure are now aligned.

17. Appendix — Light Code Snippets and Diagrams

Maintenance SQL mental model

```
SELECT * FROM jc_maintenance_details WHERE jc_section_no = ? ORDER BY sr_no, id;
INSERT row with next sr_no = MAX(sr_no)+1;
PATCH only changed fields;
DELETE by row id after parent ownership check;
```

Spares SQL mental model

```
SELECT * FROM jc_spares_used WHERE jc_section_no = ? ORDER BY sr_no, id;
Normalize decimal values;
Default source to CASH_PURCHASE;
Hard-delete row after authorization;
```

Date coercion

```
if field is equipment_submitted_date or equipment_received_date_actual:
    display = raw_value.slice(0, 10)
else:
    display = raw_value
```

Mark Complete freshness

```
await patchJobCardTab(id, values)
invalidateAll()
refetch()   // immediate parent detail refresh
```

Closure gate

```
data tabs: canWrite = status == IN_PROGRESS
mark-complete: canMarkComplete
closure: canVerifyClose = LIC/SA + status == COMPLETED + permission
```

17.1 Senior Engineering Retrospective

Lesson	Why it matters for next phase
Do not defer visible child-tab CRUD after schema exists	Users judge completion by workflow, not migration existence.
Date/time wire format must be UI-compatible	HTML inputs enforce strict value formats.
Invalidate is not refetch	Cache clearing alone does not guarantee immediate fresh render.
One canWrite flag is not enough for multi-state tabs	Each tab must map to its own state machine permission.

Lesson	Why it matters for next phase
Gate panels must explain blockers	Disabled buttons without hints create confusion and slow testing.

17.2 Locked Checklist for Future Regression Testing

Regression step	Expected result
Open a newly assigned Job Card and start work.	Expect IN_PROGRESS status and simple tabs editable.
Add three maintenance rows.	Expect sr_no ordering and persistence after refresh.
Edit maintenance observation and action fields.	Expect on-blur PATCH and persisted values.
Add two spare rows with different sources.	Expect enum source persistence.
Enter decimal quantity and cost values.	Expect numeric values preserved after reload.
Save equipment submitted date.	Expect date input still shows saved date after refresh.
Clear a date field.	Expect backend field becomes NULL and UI blank.
Type fewer than 20 observation characters.	Expect Mark Complete gate shows exact deficit.
Type 20+ observation characters and save.	Expect gate turns green without waiting 15 seconds.
Upload required/certificate document.	Expect document gate turns green.
Complete all tasks.	Expect task gate shows x/y complete.
Submit Mark Complete.	Expect status moves to COMPLETED.
Login as LIC and open Closure tab.	Expect form editable.
Fill closure review and customer acknowledgment.	Expect Close button enables.
Submit closure.	Expect status moves to VERIFIED_CLOSED and card becomes read-only.

18. Extended DS Addendum — Hotfix Deep Documentation

WHY THIS ADDENDUM EXISTS

A hotfix is not only a patch. For CMCMIS, every hotfix becomes future developer memory. This addendum expands each delivered fix into reusable engineering reasoning so the next phase can continue without re-discovering the same bugs.

18.1 Maintenance Repo Deep Dive

DS EXPLANATION

The repo layer knows jc_maintenance_details and nothing else. It lists rows by jc_section_no, computes sr_no locally as max+1, updates only editable fields, and hard-deletes one row by id. This keeps database behavior stable and predictable.

Technical breakdown

Item	Explanation
listForJc	Reads all rows for a single Job Card, ordered by sr_no then id.
findById	Confirms the row exists and belongs to the requested parent before mutation.
insertRow	Calculates next sr_no, truncates large strings safely, inserts child row.
updateRow	Builds dynamic SET clause only for provided editable fields.
deleteRow	Hard-deletes the child row as per Q-5 rule.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.2 Maintenance Service Deep Dive

> *DS EXPLANATION*
>
> The service layer enforces meaning. It checks parent Job Card existence, legacy read-only status, ownership, and role. The repo can modify rows, but the service decides whether a user is allowed to request that modification.

Technical breakdown

Item	Explanation
loadAndAuthorize	Single choke point for parent card load + mutation authorization.
listRows	Safe read path for authorized detail readers.
addRow	Engineer/LIC can add defect detail rows.
updateRow	Prevents row-id tampering by verifying jc_section_no.
deleteRow	Hard-delete only after parent and row validation.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.3 Maintenance Controller and Routes

DS EXPLANATION

The controller is intentionally thin: parse rowId, pass req.user and req.body to service, return JSON envelope. The routes apply authentication, permission, and validation before business code runs.

Technical breakdown

Item	Explanation
GET /maintenance-rows	read-detail only
POST /maintenance-rows	update-tasks permission + service-level ownership
PATCH /maintenance-rows/:rowId	partial validation + ownership
DELETE /maintenance-rows/:rowId	explicit hard-delete path

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.4 Spares Repo Deep Dive

> *DS EXPLANATION*
>
> *The spares repo mirrors the maintenance repo but handles additional field types: source enum, quantity decimal, cost decimal, and optional part metadata. This mirror pattern keeps developer mental load low.*

Technical breakdown

Item	Explanation
source default	CASH_PURCHASE when the UI posts an empty row.
quantity/cost	Converted to Number or NULL before SQL update.
part fields	Truncated safely and stored as nullable text.
sr_no	Computed independently per Job Card.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.5 Spares Service Deep Dive

> DS EXPLANATION
>
> Spares service uses the same authorization doctrine as maintenance. This protects the child data while allowing the assigned engineer to work without LIC intervention.

Technical breakdown

Item	Explanation
row belongs to card	Prevents editing a spare row from another Job Card.
legacy card check	Blocks mutation on imported historical cards.
own engineer rule	Allows the actual assigned engineer to edit.
LIC/SA override	Allows supervisory correction.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.6 Spares Frontend UX

> **DS EXPLANATION**
>
> Spares UI uses a table because engineers often enter multiple small items quickly. The source dropdown commits immediately because a dropdown change is atomic. Numeric fields commit on blur to avoid unnecessary PATCH requests.

Technical breakdown

Item	Explanation
Add Spare	POST empty row, refetch table.
Source dropdown	PATCH immediately on change.
Quantity/cost	Use number input with step 0.01.
Delete	Confirm then hard-delete.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.7 Maintenance Frontend UX

> **DS EXPLANATION**
>
> Maintenance uses multiline textareas because defect description, observation, action taken, and remarks are narrative fields. The row saves field-by-field on blur, which is faster than forcing a modal.

Technical breakdown

Item	Explanation
Defect Description	Required conceptually; placeholder row is created then edited.
Observation	Engineer diagnostic notes.
Action Taken	Repair or calibration step performed.
Remarks	Extra context or outcome notes.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.8 Date Bug Deep Dive

> *DS EXPLANATION*
>
> This bug is subtle: the backend can be perfectly correct and still the browser can appear wrong. The correct mental model is not persistence failure; it is wire-format/UI-format mismatch.

Technical breakdown

Item	Explanation
Server DATETIME	ISO string with time zone suffix.
Browser date input	Strictly YYYY-MM-DD.
Symptoms	Input appears blank after save.
Fix location	Shared form coercion hook, not backend schema.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.9 Date Bug Regression Rules

DS EXPLANATION

Any future date-like field must declare whether it is DATE-only or DATETIME stored-as-date. If it reaches input[type=date], the displayed value must be exactly 10 characters in YYYY-MM-DD format.

Technical breakdown

Item	Explanation
Rule 1	Do not bind ISO datetime directly to date input.
Rule 2	Do not strip actual time fields that are rendered by datetime-local.
Rule 3	Treat empty string as NULL for persistence.
Rule 4	Document exceptions in the shared form hook.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.10 Mark Complete Gate Deep Dive

> **DS EXPLANATION**
>
> The Mark Complete panel is a trust surface. It should tell the engineer what is missing, not only disable a button. The hotfix improves both data freshness and human clarity.

Technical breakdown

Item	Explanation
Task gate	Shows x/y complete.
Observation gate	Shows current character count vs 20.
Document gate	Shows what doc type is missing.
Jump links	Move directly to the tab where the missing item can be fixed.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.11 Mark Complete Cache Discipline

> **DS EXPLANATION**
>
> Invalidation only makes old cache invalid; it does not guarantee the current render has fresh data. The hotfix uses refetch() immediately after successful save.

Technical breakdown

Item	Explanation
Before	invalidateAll() then wait for poll.
After	invalidateAll() + refetch() immediately.
Benefit	Gate panel responds without reload.
Risk removed	User no longer sees disabled button after doing the right action.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.12 Closure Gate Deep Dive

> **DS EXPLANATION**
>
> Closure is a different lifecycle stage from editing. The generic canWrite flag intentionally required IN_PROGRESS for data tabs. Reusing it in Closure caused a false-negative permission condition.

Technical breakdown

Item	Explanation
Data tabs	IN_PROGRESS only.
Mark Complete	IN_PROGRESS with complete permission.
Closure	COMPLETED with verify-close permission.
Closed card	VERIFIED_CLOSED read-only.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.13 Orchestrator Responsibility

> **DS EXPLANATION**
>
> JobCardDetail.jsx is not only a router. It is the policy boundary for which tab exists and whether that tab may write. The hotfix makes that boundary explicit.

Technical breakdown

Item	Explanation
visibleKeys	Controls which tabs are shown.
activeTabCanWrite	Controls whether active tab can mutate.
tabProps	Passes resolved write flag to tab.
future proof	New tabs can add their own write gates.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.14 API Wrapper Discipline

> **DS EXPLANATION**
>
> The frontend api layer prevents route strings from being scattered across components. This matters because future backend route changes should be patched once.

Technical breakdown

Item	Explanation
fetchMaintenanceRows	GET maintenance rows
addMaintenanceRow	POST child row
patchMaintenanceRow	PATCH one child row
deleteMaintenanceRow	DELETE one child row
same for spares	Parallel wrapper set.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.15 Hook Discipline

> **DS EXPLANATION**
>
> The hooks own cache, loading, error, and refetch. The tab component owns local user interaction. This separation makes bugs easier to isolate.

Technical breakdown

Item	Explanation
useMaintenanceRows	Child-row fetch/cache/refetch.
useSparesRows	Child-row fetch/cache/refetch.
useJobCardDetail	Parent detail payload.
useAutoSave	Simple tab save behavior.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.16 Smoke Matrix Philosophy

> **DS EXPLANATION**
>
> A smoke test should exercise the exact thing that failed in the browser. The hotfix smoke does not only check endpoints exist; it adds, reads, patches, deletes, and verifies date round-trip semantics.

Technical breakdown

Item	Explanation
H1–H3	Maintenance CRUD
H4–H6	Spares CRUD
H7–H9	Date persistence behavior
FE build	Bundle integrity

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.17 Production Risk Reduction

> **DS EXPLANATION**
>
> This hotfix reduces operational risk because Job Card completion now has fewer hidden dead-ends. Engineers can fill all fields, correct mistakes, upload documents, pass gates, complete the card, and LIC can close it.

Technical breakdown

Item	Explanation
Risk removed	Stubs in production flow.
Risk removed	Misleading empty date inputs.
Risk removed	Disabled button caused by stale data.
Risk removed	Closure unusable in completed state.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.18 Future Developer Warning

> **DS EXPLANATION**
>
> Do not introduce future tabs using the generic canWrite blindly. Always ask: which lifecycle status owns this tab? Which role can mutate it? Which permission should gate it?

Technical breakdown

Item	Explanation
Question 1	What state should this tab be writable in?
Question 2	Which role owns the action?
Question 3	Is this a parent field or child-row field?
Question 4	Does save need immediate refetch?

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

18.19 Next Phase Carry-Forward

> **DS EXPLANATION**
>
> Reports, exports, PDF generation, and audit viewers must follow the same pattern: backend route contract, service-level authorization, frontend hook, clear user blocking messages, and deterministic smoke.

Technical breakdown

Item	Explanation
Reports	Use filters and export state carefully.
PDF	Do not enable download until document prerequisites exist.
Audit viewer	Read-only, permission-aware.
Exports	Role-filtered and query-limited.

Locked checklist

Senior check	Expected answer
Does this change preserve existing schema doctrine?	Yes, it uses existing Phase 9 tables and nested resources.
Does this change protect legacy Job Cards?	Yes, service-level checks prevent mutation of legacy read-only rows.
Does this change improve real user workflow?	Yes, it removes a direct blocker from engineer/LIC completion path.
Does this change have an acceptance signal?	Yes, smoke checks and FE build verified the behavior.

PHASE 9 HOTFIX

SEALED · LOCKED · READY FOR NEXT PHASE

FINAL DS SEAL

This document records the completed Phase 9 hotfix: Maintenance Details CRUD, Spares Used CRUD, date-field display persistence, Mark Complete stale-data refetch, and Closure status-gate correction. The phase is now operationally sealed for this workflow slice.

Seal Field	Seal Value
Prepared for	DS / CMCMIS_SIMPLIFIED production-grade workflow
Status	SEALED after smoke verification and build verification
Next recommended phase	Reports/Exports or Phase 10 hardening, depending on project sequence